Object Oriented Programming using C++
Classes and Objects

Y. T. Ching
Department of Computer Science
NYCU

- Structure Programming
  - C, PASCAL
  - Design a program by designing tasks
  - DrawLine(...), DrawCircle(...), Draw(LINE,...) Draw(CIRCLE,...).
- Object-Oriented Programming
  - Designed a program by designing objects
  - Objects for Line and Circle; Line.Draw(), Circle.Draw()

OOP technology implements the important concept

- Encapsulation
- Inheritance
- Polymorphism

Encapsulation

- Encapsulation: the concept of binding together data and functions into one capsule or object.
- The concept is also known as data abstraction.
- The implementation enables data hiding: data in the object is hidden from the rest of the world. The data can be accessed through the interface functions.

Inheritance

- Inheritance: one of the most important tools in the implementation of code reusability.
- When using inheritance, new code can be derived or inherited from existing code.

Polymorphism

- Concept of using the same functions on different types of objects.
- Similar to function overload, but useful when the cases are "dynamic".

- class: abstract characterization of a set of objects.
  - example
    - a set of objects that consist of figures to draw.
    - figure class: the class all these objects belong.
  - class defines variables (data) and methods (operations) common to a set of objects.
  - class: prototype or generator of a set of objects.

- examples for abstract data type
  - a circle class:
    - Data, the representation of a circle, center and radius
    - methods: draw, move, erase,...
    - manipulates a circle by the methods without knowing the underlying data structure for the circle.
  - a priority queue
    - implementation of priority queue: array, list, heap,..
    - methods: delete min, insert arbitrary, ...
    - manipulates a priority queue by the methods.

- To access any member variable, use the membership operator (dot . operator).

```
speaker.impedance = 8;
amplifier.quantity = 1;
cout << amplifier.description;
```

- An example of C++ class

```
class string {
  char val[80];
public:
  void store (char *);
  int length();
};
```

- `string` class consists of data `val` and method `store (char *)` and `length()`.
- `string s, t`
- `s` and `t` are "instances" of `string`

- To access a member (data or method), C++ follows C's syntax
  - `s` an object, `x` a data member of `s`, `f` a method of `s`.
  - `s.x;, s.f(arguments);`
  - `ptr` a pointer to `s`
  - `ptr -> x;`
  - `ptr -> f(arguments);`

- Inheritance
  - produces a new class by deriving the new class from an already existing class.
  - derived class, inherited from the original class
  - in C++
    - derived class, subclass
    - base class, superclass
  - a derived class inherits all the data and methods from the existing class.
  - a derived class may add additional data or methods.
  - a derived class can redefine inherited method.
  - reusability

- An example

```
class pen {
  int x;
  int y;
  int status;
public:
  void set_status(int);
  void set_location(int,int);
};
```

- `pen p`, `p` an object of `pen`.
- `p.set_location(x,y);` a message, locate `p` at `(x,y)`.
- `p.set_status(1);` a message, set pen `p` on.

- hardware upgrade, we have a color pen. rewrite "colored_pen" or derive from "pen".

```
class colored_pen :  public pen {
  int color;
public:
  void set_color(int);
};
```

- colored_pen has all that pen (the base class) has, x,y,status,set_status,set_location
- colored_pen has some more, color, set_color.

Polymorphism

- poly- many, morphism- forms
- polymorphism in OO programming: identically-named methods that have different behavior depending on the type of object reference.
- Example
  - a class `shape` and two subclasses `circle` and `line`
  - `shape` has a method `draw`, and redefined in both subclasses.

- `shape* composite_fig[100];`
- In C++, we can assign a pointer to a base class object pointing to a derived class object.
- If we have a program to take user inputs, then draw many circles and lines.
- We don't know what `composite_fig[i]` has because it depends on user input.
- What polymorphism helps: if the i'th pointer points a `line` (`circle`), `line`'s (`circle`'s) `draw` will be invoked.
- `for (i=0;i<100;i++) composite_fig[i]->draw();`

Class

- Class: the foundation of OOP, very similar to a struct.

```
class class_name { // Name starts with upper case
private:
  // private variable and functions
public:
  // public variable and function
} object_list; // optional
```

- Class members: functions and variables declared inside the class.
  - Member variables, also called data members.
  - Member functions.
- C++ uses classes to create objects. An object is an instance of a class.
  - class data members can be viewed as the object's attributes or properties.
  - member functions are its behavior or methods.
  - Spacecraft is an object with attributes such as velocity, acceleration, and position. And functions that manipulate the spacecraft's attributes describe its behavior.

- Accessing class member
- member can be data or function
  - private: member can be accessed by the members of the same class.
  - public: classes member can be accessed anywhere.
  - protected: needed in inheritance.
- Data members are usually defined as private members.
- Functions (methods) are usually defined as public, serve as "interface",
- outside world uses methods to access or manipulate private data.
- In C++, the default access specifier is private.

```
class Jet {
  float acc, vel, dis;
  float getTime() {
    return vel/acc;
  }
public:
  void setValue(float x, float y) {
    acc=x; vel=y;
  }
  float getDisplacement() {
    return (vel*getTime())/2;
  }
};
```

```
int main()
{
  Jet plane;
  plane.setValue(40,65);
  cout << "The time during which the plane ";
  cout << "is being accelerated = ";
  cout <<plane.getTime(); // ERROR;
  cout << "\n The plane's displacement = ";
  cout << plane.getDisplacement();
  return 0;
}
```

Prog4_3

```
#include <iostream>
#include <cmath>
using namespace std;
class Resister {
private:
  int band1, band2, band3, band4;
  int resistance;
public:
  void getValue();
  void calResistance(){
  resistance=(((band1*10+band2)*pow(10,band3));
  }
  void printResistance();
};
```

```
void Resistor::getValues()
{
  cout<<"\t\tCOLORS:";
  cout<<"\n\n 0==BLACK ...."<<endl;
............
  cout<<"10==GOLD 11==SILVER .." << endl;
  cout <<"Please select first band (1-9)==> ";
  cin >> band1;
  // then input all the other bands
  // band2, band3, band4
}
```

```
void Resistor::printResistance()
{
  char *tolerance;
  calResistance();
  if (band==10) tolerane = "+/- 5%";
  if (band==11) tolerane = "+/- 10%";
  if (band==12) tolerane = "+/- 20%";
  cout<<"\n\n The resistance is "
    <<resistance<<" ohms";
  cout<<" with "<< tolerance<<"
    tolerance." <<endl;
}
```

```
int main()
{
  Resistor res;
  res.getValues();
  res.printResistance();
  return 0;
}
```

Complex Example

```
#include <stdio.h>
/* structure definition */
typedef struct
{
  double real, imag;
} complex;

/* prototype */
int main(void);
complex cplx_set(double r, double i);
void cplx_print (complex c);
complex cplx_add (complex c1, complex c2);
complex cplx_sub (complex c1, complex c2);
```

```
int main (void)
{
  complex a, b, c, d;
  a = cplx_set(1.0,1.0);
  b = cplx_set(2.0,2.0);
  c = cplx_add (a,b);
  d = cplx_add (b, cplx_sub (c,a));
  printf ("c = ");
  cplx_print (d);
  printf ("and ");
  printf ("d = ");
  cplx_print (d);
  printf ("\n");
}
```

```
complex cplx_set(double r, double i);
{
  complex temp;
  temp.real = r;
  temp.imag = i;
  return temp;
}

void cplx_print (complex c)
{
  printf ("(%g, %g)", c.real, c.imag);
}
```

```
complex cplx_add (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real + c2.real;
  temp.imag = c1.imag + c2.imag;
  return temp;
}
complex cplx_sub (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real - c2.real;
  temp.imag = c1.imag - c2.imag;
  return temp;
}
```

Now modify the program in C++ syntax

```
#include <stdio.h>
/* structure definition */
typedef struct
{
  double real, imag;
} complex;
/* prototype */
int main(void);
complex cplx_set(double r, double i);
void cplx_print (complex c);
complex operator + (complex c1, complex c2);
complex operator - (complex c1, complex c2);
```

```
/* main program */
int main (void)
{
  complex a, b, c, d;
  a = cplx_set(1.0,1.0);
  b = cplx_set(2.0,2.0);
  c = a + b;
  d = b + c - a;
  printf ("c = ");
  cplx_print (d);
  printf ("and ");
  printf ("d = ");
  cplx_print (d);
  printf ("\n");
}
```

```
complex operator + (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real + c2.real;
  temp.imag = c1.imag + c2.imag;
  return temp;
}

complex operator - (complex c1, complex c2)
{
  complex temp;
  temp.real = c1.real - c2.real;
  temp.imag = c1.imag - c2.imag;
  return temp;
}
```

Now the C++ Code

```
#include <stdio.h>
class complex
{
  private:
    double real, imag;
  public:
    void set (double r, double i=0);
    void print();
    complex operator + (complex c);
    complex operator - (complex c);
}
int main (void); // prototype
```

```
int main(void)
{
  complex a, b, c, d;
  a.set(1.0,1.0);
  b.set(2.0,2.0);
  c = a + b;
  d = b + c - a;
  printf ("c = ");
  c.print ();
  printf ("and ");
  printf ("d = ");
  d.print ();
  printf ("\n");
}
```

```
void complex::set(r,i)
{
  real = r;
  imag = i;
}

void complex::print()
{
  printf ("(%g,%g)", real, imag);
}
```

```
complex complex::operator + (complex c)
{
  complex temp;
  temp.real = real + c.real;
  temp.imag = imag + c.imag;
  return temp;
}
complex complex::operator - (complex c)
{
  complex temp;
  temp.real = real - c.real;
  temp.imag = imag - c.imag;
  return temp;
}
```

A Stack Class

```
const int MaxStack = 10000;
const char EmptyFlag = '\0';
class Stack {
  enum {FullStack=MaxStack, EmptyStack=-1};
  enum {False=0, True=1};
  char items [MaxStack];
  int top;
```

```
public:
  // methods
  void init();        // set top
  void push(char);    // add an item
  char pop();         // remove an item
  int empty();        // no elements?
  int full();         // too many elements?
  void dump_stack();  // top to bottom
};
```

```
void Stack::init()
{
  top=EmptyStack;
}

void Stack::push(char c)
{
  if (full()) return;
  item[++top]=c;
}
```

```
char Stack::pop()
{
  if (empty()) return EmptyFlag;
  else return item[top--];
}

int Stack::full()
{
  if (top+1==FullStack) {
    cerr << "Stack full at "
      << FullStack << endl;
    return True;
  }
  else return False;
}
```

```
int Stack::empty()
{
  if (top==EmptyStack) {
  cerr << "Stack empty" << endl;
  return True;
  }
  else return False;
}

void stack::dump_stack()
{
  for (int i=top; i>=0; i--)
    cout << item[i]<<endl;
}
```

```
a code slice
Stack s1;
s1.init();
s1.push('a');
s1.push('b');
cout<<s1.pop();
s1.push('c');
cout<<s1.pop();
s1.push('b');
s1.push('c');
s1.dump_stack();
```

- The `stack` class
  - declaring a class does not allocate space.
  - needs a method to initialize a newly created object (`init` in `stack` class).
  - methods (`push`, `pop`) are `public`, can be accessed by code outside.
  - data, `items[]` and `top` are private, cannot be accessed by code outside, but can be accessed by its own methods.
  - manipulates the private data through the public method.

```
void f()
{
  Stack s;
  s.init();
  s.push(‘a’); // OK
  cout << s.item[0] << endl; // an error.
}
```

- The `Stack` exampls does not have a constructor.
- constructor and destructor are two special types of methods.
- invoke `init` after `s1` is created. If not, the `top` of `s1` contains junk.
- A reasonable approach: have a constructor that does the initialization automatically.

```
class Stack {
  enum {FullStack=MaxStack, EmptyStack=-1};
  enum {False=0, True=1};
  char items[MaxStack];
  int top;
public:
  Stack(); // the constructor
  void push (char); // add an item
  char pop (); // remove an item
  int empty(); // no elements?
  int full(); // too many elements?
  void dump_stack(); top to bottom
};

Stack::Stack()
{ top = EmptyStack; }

Stack s1; //create s1, constructor invoked
```

- Initializing variables before they are used is a common programming practice.
- Constructor: Can serve as a function to do initialization.
- A constructor
  - In C++, a class must have at least a constructor and may have more than one constructor.
  - constructor can be user-defined or compiler-generated.
  - It is never explicitly called. A constructor function is automatically called whenever an object is declared.
  - It has the same name as the class, no return type, not even `void`
  - It can have arguments, including default arguments.
  - Constructor should be `public`, so they can be called outside the class.
  - Constructor cannot be inherited, each class should have it own constructors.

- We cannot initialize class data members within the class.

```
class Battery {
  float voltage=12.0;
  float capacity;
}
```

- It should be initialized in the constructor

```
Battery ()
{
  voltage = 12.0;
  capacity = 30.0;
}
```

- If there is a line `Battery bt;`
- `Battery ()` is invoked.
- we have
  - `bt.voltage == 12`, and
  - `bt.capacity == 30`
- What if we want values other than 12 and 30?
- We need another constructor (we can have more than one constructor).

```
Battery (float v, float c)
{
  voltage = v;
  capacity=c;
}
Battery bt(12.0, 30.0);
```

Or

```
Battery (float v=12.0, float c=30.0)
{
  voltage = v;
  capacity=c;
}
Battery bt1; or Battery b2(5.0, 20.0);
```

A class may have more than one constructor

```
class Color {
  float red;
  float green;
  float blue;
public:
  Color () {red=green=blue=0.0;}
    // 1st constructor
  Color (float, float, float); //2nd constructor
};

Color::Color(float r, float g, float b)
{ red=r; green=g; blue=b; }

Color c1; //invoke 1st constructor
Color c2(1.0,0.5,0.0); //invoke 2nd constructor
```

- parameterized constructor has initial values
  - `Color c2(1.0);`
  - `c2.red = 1.0` given by the argument.
  - `c2.green=0.5`, and `c2.blue=0.0` are as initial values.
- initial values are given from left to right.
  - `// The following is an error`
  - `Color::Color(float r =0.5 , float g, float b=0.0)`
    `{ red = r; green = g; blue = b;}`

- If there is no constructor, the compiler will generate a default one `(Color())`.
- If there is a constructor with arguments (not a default constructor). The compiler will not generate a default compiler,
- The user needs to provide arguments as specified when constructing an object.

```
class C1 {
  int iC1;
  float fC1;
public:
  // initialization in constructor body
  C1 (int a1, float a2) {iC1=a1; fC1=a2;}
};

// initialization in constructor header
C1 (int a1, float a2) :  iC1(a1), fC1(a2) {}
// initialization is done in the header, so
// empty constructor body

// mixed initialization
C1 (int a1, float a2) :  iC1 (a1) fC3=a2;
```

Header initialization is required if a class has a data member 1. `const` 2. object in some other class.

- Examples need header initialization

```
class C {
  const int x; // a const data member
  float y;
public:
  C(int a1, float a2) :  x(a1), y(a2) {}
} // x need header initialization, y need not.
class X {
  char moniker[20];
public:
  X(char* m){strcpy (moniker, m);}
};
class Y{
  int num;
  X x;
public:
  Y(int a1, char* a2):num(a1),x(a2) {}
};
```

- Copy constructor
  - default constructor, no argument.
  - constructors other than the default constructors must have an argument(s).
  - The type of the argument cannot be the same as the class. A class C constructor must not take an argument of type C.
  - Type of the argument can be a reference to the class- a copy constructor.
  - constructs a new object by copying an existing one.
  - compiler gives one if the programmer didn't provide one.

```
class Color{
  float red; float green; float blue;
public:
  Color() {red=green=blue=0.0;}
  Color(Color); // an error
};

class Color{
  float red; float green; float blue;
public:
  Color() {red=green=blue=0.0;}
  Color (float, float, float);
  Color (Color &); // Copy
};

Color::Color(Color& c)
{red=c.red; green=c.green; blue=c.blue;}
```

- Convert Constructor
  - another one-parameter constructor (other than `C&`).
  - Convert type `t` to type `C`.

```
class Clock {
  int hour;
  int min;
  int ap;
public:
  Clock() {hour=12; min=0; ap=0;}
  Clock (int);
  void print_time();
};

Clock::Clock (int time)
{
  min = time % 100;
  hour = time / 100;
  if (hour>12) {
    hour -= 12;
    ap = 1;
  }
  else ap=0;
}
```

```
void Clock::print_time()
{
  cout << setfill('0')<<setw(2) << hour
  << ':'  << setw(2) << min
  << setfill (' ');
  if (ap) cout << " PM";
  else cout << " AM";
  cout << endl;
}
// print out xx:xx XX
// 12:00 AM

// Clock::Clock(int time) convert time
// to Clock.
```

- if `c` is a `Clock` object,
  - `c = Clock (1308); c.print_time(); // get 01:08 PM`
  - `c = (Clock) 123;` a cast, `123` is convert to a `Clock` object by convert constructor, then copy to `c`.
  - `c = 1155;` implicit type conversion same as the above, the compiler uses the convert constructor convert 1155 to a `Clock` then copy to `c`.

- Two tasks of the constructor
  - initialization,
  - dynamically allocates storage and then initializes the storage.

```
class C {
  char* text;
  ...
public:
  C();
};

C:C()
{
  text = new char[20]; // allocate
  strcpy (text, "North By Northwest");
}

C c; // create a C object
// allocate space char [20]
// and initialization
```

- destructor:
  - a special method like a constructor that the programmer writes but does not call.
  - automatically invoked whenever an object is destroyed.
  - free memory allocated in the constructor.
  - No argument, no return value.
  - `Color::~Color()`

a piece of C code

```
typedef struct big_string {
  char* string;
  int size;
} BigStr;

#define OneBillion (1000000000)
void big_grabage()
{
  BigStr bs;
  bs.size = OneBillion;
  bs.string =
    (char*) calloc(bs.size,sizeof(char));
  /* process bs.string but forget to free it */
  ...
}
```

big garbage after execution of the function

a C++ code

```
const long OneBillion = 1000000000;
class BigStr {
  char* string;
  long size;
public:
  BigStr();
  ~BigStr() {delete[] string;}
};
BigStr::BigStr()
{
  string = new char [OneBillion +1];
  for (long i=0;i<OneBillion;i++)
    string[i] = '$';
  string[OneBillion] = '\0';
  size = OneBillion;
}
```

A Zipcode class

- a class to manipulate zip codes of different lengths (5- or 9-digit codes).
- constructors allow zip codes to be generated from either character or integer.

```
#include <stdio.h>
#include <iostream.h>
#include <string.h>

const int MinZip = 5; //60607
const int BigZip = 10;
const int MaxZip = 32;
const int InitChar = '?';
const int Hyphen = '-';
const int SuffixLen = 4; // 60607-1234
```

```
class ZipC {
  char* code;
public:
  ZipC();
  ZipC(const char*);
  ZipC(const unsigned long);
  ~Zip();
  void write () {cout<< code << endl;}
  void expand (const char *);
};

ZipC::ZipC()
{
  code = new char[MinZip+1];
  for (int i = 0;i<MinZip;i++)
    code[i]=InitChar;
  code[i] = '\0';
}
```

```
ZipC::ZipC (const char* zipstr)
{
  int len = (strlen(zipstr)<MaxZip) ?
  strlen(zipstr) :  MaxZip;
  code = new char [len+1];
  code[len]='\0;
}

ZipC::ZipC(const unsigned long zipnum)
{
  char buffer[BigZip+1];
  sprintf (buffer, "%d*%d",MinZip,zipnum);
  buffer[MinZip] = '\0';
  code = new char[strlen(buffer)+1];
  strcpy (code, buffer);
}
```

```
ZipC::~ZipC()
{
  delete [] code; }

void ZipC::expand (const char* suffix)
{
  char temp[BigZip+1];
  char previous[MinZip+1];
  if (strlen(code)!=MinZip ||
      strlen(suffix)!=SuffixLen)
    return;
  strcpy (previous,code);
  delete[] code;
  code=new char [BigZip+1];
  sprintf (code,"%s%c%s",
    previous,Hyphen,suffix);
}
```

- C++ provides a special member function called the destructor function.
- Destructor is automatically invoked when an object is destroyed.
- Needed, especially when there are dynamically allocated memory spaces.
- There are many ways to construct an object (more than one constructor), but there is only one way to destroy an object (only one destructor).

- Properties of destructor functions
  - Its name is the tilde (~) character followed by the class name.
  - It cannot have a return type.
  - It cannot have arguments.
  - It is automatically called when an object goes out of scope.

```
class ErrMessage {
  char *mes;
public:
  Errmessage (char *em) {
    mes=new char[strlen(em)];
    strcpy (mes, em);
  }
    // other member functions
};
```

There should be a destructor

```
~ErrMessage(){delete [] mes;}
```

A battery example

```
class Battery {
private:
  float voltage;
  float capacity;
public:
  Battery() {
    voltage=12;
    capacity=20;
  }

  battery(float v, float cp) {
    voltage=v;
    capacity=cp;
  }
```

```
  void getValues() {
    cout<<setiosflags(ios::fixed)<<setprecision(1);
    cout<<"voltage= "<<voltage<<" [V]"<<endl;
    cout<<"Capacity = "<<capacity<<" [Ah]"<<endl;
  }

  float getHours(float imp) {
    float current=voltage/imp;
    return capacity/current;
  }

  ~ Battery() {cout<<"\n Battery discharged!!";}
};
```

```
int main()
{
  float imp=100;
  {
    Battery b1;
    cout<<"First battery:  "<<endl;
    b1.getValue();
    cout<<"Device can be powered "<<b1.getHours(imp);
    cout<<" hours,"<<endl;
  } //Calls destructor to destroy b1
```

```
  Battery b2(12,10);
  cout<<"\n\nSecond battery:  "<<endl;
  b2.getValues();
  cout<<"Device can be power "<<b2.getHours(imp);
  cout<<" hours."<<endl;
  return 0;
}
```

Copy Constructor

- A constructor is called when a new object is initialized by copying an existing object.
- Situations occur when
  - An object is used to initialize another object in a declaration statement.
  - An object is passed to a function.
  - An object is returned from a function.

- Objects passing to and returning from functions
- by pointer, by reference, and by value
- If it is the case "by value", a copy of the object is going to be created in the function.
- The "copy constructor" shall be invoked.
- Prog 5_1 and Prog5_2,
- look at how many times constructor and destructor are invoked.

Program 5_1

```
#include <iostream>
using namespace std;
int bcount=0; // A global variable
class Box {
  float side1, side2, side3;
public:
  Box(float s1, float s2, float s3)
  {
    side1=s1; side2=s2; side3=s3;
    bcount ++;
    cout <<"bcount "<<bcount<<flush<<endl;
  }
```

```
  Box(Box & b)
  {
    side1=b.side1;
    side2=b.side2;
    side3=b.side3;
    bcount++;
  }
  ~ Box()
  {
    bcount--;
    cout<<"\t\tBox destroyed!\n";
  }
  float getVolume()
  {
    return side1*side2*side3;
  }
};
```

```
float addBoxes(Box, Box); // A function, prototype
int main(int argc, char * argv[])
{
  Box a(1,2,3), b(2,3,4);
  cout<<bcount<<" boxes built so far in main().\n";
  cout<<"\t\tTotal volume = "<<addBoxes(a,b)<<endl;
  cout<<bcount<<" boxes built so far.\n";
  return 0;
}
float addBoxes(Box b1, Box b2)
{
  cout <<"\t\t in the addBoxes Adding boxes:\n";
  cout<<"\t\t"<<bcount<<" boxes built so far.\n";
  return b1.getVolume()+b2.getVolume();
}
```

Program 5_2

```
#include <iostream>
using namespace std;
int bcount=0;
class Box {
  float side1, side2, side3;
public:
  Box(float s1, float s2, float s3)
  {
    side1=s1; side2=s2; side3=s3;
    bcount ++;
  }
  ~ Box()
  {
    bcount--;
    cout<<"\t\tBox destroyed!\n";
  }
```

```
  float getVolume()
  {
    return side1*side2*side3;
  }
};
```

```
float addBoxes(Box &, Box &);
int main(int argc, char * argv[])
{
  Box a(1,2,3), b(2,3,4);
  cout<<bcount<<" boxes built so far.\n";
  cout<<"\t\tTotal volume = "<<addBoxes(a,b)<<endl;
  cout<<bcount<<" boxes built so far.\n";
  return 0;
}
float addBoxes(Box &b1, Box &b2)
{
  cout <<"\t\tAdding boxes:\n";
  cout<<"\t\t"<<bcount<<" boxes built so far.\n";
  return b1.getVolume()+b2.getVolume();
}
```

- Returning an object from a function, a temporary object is constructed in memory to store the return value
- then return that value to the program statement that calls the function.
- After the value is returned, class destructor is invoked to destroy the object
- the control is then returned back to the calling function.
- Constructing a temporary object is done by calling the "copy constructor"
- Prog5_3

Program 5_3

```
#include <iostream>
using namespace std;
class Pixel {
  int x,y;
public:
  Pixel()
  {
    x=0;y=0;
    cout << "pixel constructed \n";
  }

  ~ Pixel(){cout<<"\t\tPixel destroyed!"<<endl;}

  void setXY(int x1, int y1) {x=x1; y=y1;}
```

```
  void getCoord() {
    cout<<"Pixel's coordinate:"<<endl;
    cout<<"X="<<x<<" Y="<<y<<endl;
  }

  Pixel move_10(Pixel t) {
    t.x = t.x + 10;
    t.y = t.y + 10;
    return t;
  }
};
```

```
Pixel setCoord()
{
  int x1 ,y1;
  cout<<"Enter x and y coordinates =>";
  cin >>x1>>y1;
  Pixel temp;
  temp.setXY(x1,y1);
  return temp;
}
```

```
int main(int argc, char * argv[])
{
  Pixel p1, p2;
  p1=setCoord();
  p1.getCoord();
  p2=p1.move_10(p1);
  p2.getCoord();
  p1.getCoord();
  return 0;
}
```

```
Pixel   p1;          // Default constructor
Pixel   p2=p1;       // Copy constructor
Pixel   P3(p2);      // Copy constructor
fun1(p1);            // Copy constructor
p2=fun2()            // Copy constructor
p3=p2                // NOT Copy constructor
```

General format for defining a copy constructor

```
class_name (const class_name & object_name)
{
// Body of the copy constructor
}
```

Prog5_4

Program 5_4

```
#include <iostream>
using namespace std;
class Pixel {
  int x,y;
public:
  Pixel(int a, int b)
  {
    x=a; y=b;
    cout<<"\tNormal Constructor" <<endl;
  }
  Pixel(const Pixel &p)
  {
    x=p.x; y=p.y;
    cout<<"\tCopy Constructor"<<endl;
  }
```

```
  ~ Pixel()cout<<"\tDestructor"<<endl;
  void setX(int x1) {x=x1;}
  void setY(int y1) {y=y1;}
  void showXY(){cout<<"X="<<x<<" Y="<<y<<endl;}
};

 Pixel center(Pixel tp)
{
  tp.setX(512);
  tp.setY(512);
  return tp;
}
```

```
int main(int argc, char * argv[])
{
  Pixel p1(10,20);
  p1.showXY();
  Pixel p2=p1;
  p2.showXY();
  p2=center(p1);
  p2.showXY();
  return 0;
}
```

- There must be a copy constructor
- If a class does not have an explicit copy constructor definition, C++ compiler creates a default copy constructor.
- The default copy constructor makes an identical (bit-by-bit) copy of the object.
- default copy constructor works fine if there are no pointers in the data objects.
- Objects contain pointers for dynamic memory allocation; bit-by-bit copy constructor causes errors.
- `p2` constructed by copying `p1`, and there are pointers in the object. Pointers in `p2` and `p1` point to smae location.
- When `p2` is destroyed, its pointer is deleted, and the pointer in `p1` is also deleted. (Prog5_5)

Program 5_5

```
#include <iostream>
using namespace std;
class Message{
  char *mes;
public:
  Message(char *m);
  Message(const Message &cm);
  ~ Message();
  void printMes(){cout<<"\tMessage:  "<<mes<<endl;}
  char * getMes(){return mes;}
};
```

```
Message::Message(char *m)
{
  cout<<"Normal Constructor Called!"<<" ";
  mes=new char[strlen(m)+1];
  cout<<"mes ==> "<<(long)mes<<endl;
  if(!mes) {
    cout<<"Allocation Error!";
    exit(1);
  }
  strcpy(mes,m);
}
```

```
Message::Message(const Message &cm)
{
  cout<<"Copy Constructor Called!"<<" ";
  mes=new char[strlen(cm.mes)+1];
  cout<<"mes ==> "<<(long)mes<<endl;
  if(!mes) {
    cout<<"Allocation Error!";
    exit(1);
  }
  strcpy(mes,cm.mes);
}

Message::~ Message()
{
  cout<<"Destructor Called"<<" ";
  cout<<"mes ==> "<<(long)mes<<" "<<endl;
  delete []mes;
}
```

```
void upperMes(Message m)
{
  int n=strlen(m.getMes());
  char * t=m.getMes();
  for(int i=0; i<n; i++)
    t[i]=toupper(t[i]);
  t[n]='\n';
  Message temp(t);
  temp.printMes();
}
```

```
int main(int argc, char* argv[])
{
  Message m1("PROG5_5");
  m1.printMes();
  Message m2("Demonstrates copy constructor!");
  Message m3=m2;
  // Message m3(m2);
  m3.printMes();
  upperMes(m3);
  m2.printMes();
  return 0;
}
```

- Organizing Files
  - header file `.hpp` `.h` and source file `.cpp` `.C` `.cc`.
  - definition of class: generally defined in a C++ header file.
  - `#include` into any other source file that will use objects of that class.
  - a problem: a file `#included` more than once.
  - each header file should contain a preprocessor statement:

```
//beginning of header
#if !defined(__complex_hpp)
#define __complex_hpp 1

// header contents

#endif
//end of header
```

- Naming
  - `NumberOfIterations`
  - `numberOfIterations`
  - `number_of_iterations`

A String Class

The header file, `string.h`

```
#ifndef StringHeaderLoaded_Version_1.0
#define StringHeaderLoaded_Version_1.0
class String {
  enum SortOrder {Asc, Desc};
  enum ErrorIO {ReadFail,WriteFail};
  char* str;
  int len;
```

```
public:
  String();
  String (const String&);
  String (const int);
  String (const char*);
  ~ String();
  int write();
  int write(FILE *);
  int read();
  int read (FILE *);
  String operator + (const String&) const;
  int operator < (const String &) const;
  int operator > (const String &) const;
  String & operator = (const String &);
  friend void sort (String*,int,int);
};
#endif
```

The source file, string.cpp

```
#include <iostream.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "string.h"
// default constructor -- create empty string
String::String()
{
  str = new char[1];
  *str = \0';
  len = 0;
}
```

```
// convert constructor
String::String (const char* cstr)
{
  str = new char[strlen(cstr)+1];
  strcpy (str, cstr);
  len = strlen(str);
}

// copy constructor
String::String(const String& strarg)
{
  // copy an existing Strarg.str into a new one
  // after first allocating sufficient storage
  strcpy (str=new char[strarg.len+1],strarg.str);
  len = strarg.len;
}
```

```
//blank string constructor
String::String(const int lenarg)
{
  str = new char[lenarg+1];
  len = lenarg;
  char* ptr = str;
  for (int i=0;i<len;i++)
    *ptr++= ' ';
  *ptr = '\0';
}
String::~ String()
{
  delete[] str;
}
```

Constructors and destructor in the example

- `String()` constructs a null String
  - `String string1;`
- `String (const char* cstr)` constructs a String by converting a C-string.
  - `String string2 ("mercy");`
- `String(const String& strarg)` constructs a String by copying an existing String.
  - `String string2("foo");`
  - `String string3(string2);`

- If there is no user-defined copy constructor, the compiler provides one copy constructor.
- The copy constructor provided by the compiler makes bit-by-bit copying.
- Suppose we have statements `String string2("foo");` and `String string3(string2);`
- The compiler's copy constructor has `str` of `string3` points to `"foo"`.
- modifying `str` of `string3` also modifies `str` of `string2`.
- But the copy constructor in the `String` example allocates another space for `str` of `string3`, and copy `"foo"` to the new space.

A note on constructor

`String::String()`	Default	`String s1;`
`String::String(char*)`	Convert	`String s2("foo");`
`String::String(int)`	Convert	`String s3(1000);`
`String::String(String&)`	Copy	`String s4(s2);`

```
// write String to std output
int String::write()
{
  return printf ("%s\n",str);
}

//write String to specific file
int String::write(FILE* outfile)
{
  if (!outfile) return WriteFail;
  else return fprintf (outfile,"%s\n",str);
}
```

```
int String::read()
{
  scanf ("%s",str);
  return len = strlen(str);
}

int String::read(FILE* infile)
{
  if (!infile) return ReadFail;
  else {
    fscanf (infile,"%s",str);
    return len = strlen(str);
  }
}
```

```
//assignment operator
String& String::operator=(const String& strarg)
{
  //check whether String assigned to itself
  if (this != &strarg) {
    delete[] str;
    len = strarg.len;
    str = new char[len+1];
    strcpy(str,strarg.str);
    return *this;
  }
  retuen *this;
}
```

- C++ keyword `this`
  - `this` is a pointer,
  - compiler sets the value of `this` to the address of the object whose function or operator member is invoked.

```
int main ()
{
  String s1 ("fred");
  s1.write();
    ...
}
```

- `this` is the pointer points to `s1`.

```
int String::write()
{
  return printf("%s\n",str);
}

int String::write()
{
  return printf ("%s\n",this->str);
}
```

The two code slices do the same thing.

```
this = ....; // An error
```

```
// <:  operator for less than comparison
int String::operator < (const String & strarg) const
{
  return strcmp (str,strarg.str)<0;
}

// >:  operatorfor greater than comparisons
int String::operator > (const String & strarg) const
{
  return strcmp (str, strarg.str)>0;
}
```

```
// +:  operator for concatenation
String String::operator + (const String& strarg) const
{
  //allocate enough strage for 2 strings + '\0'
  char* temp=new char[len+strarg.len+1];
  //copy 1st string and concatenate second
  strcpy (temp,str);
  strcat (temp, strarg.str);

  //define a variable of type String to be
  // used as a return value
  String retval(temp);

  //free temp storage
  delete[] temp;

  return retval;
}
```

```
// selection sort an array of String
void sort (String a[], int size, int order)
{ int next;
  int (String::*compare_op)(const String);
  compare_op = (order == Asc) ?
    String::operator < :  String::operator >;
  for (int i=0;i<size-1;i++) {
    next = i;
    for (int j=i+1;j<size;j++)
      if ((a[j].*compare_op)(a[next]))
        next = j;
    if (i!=next) {
      String temp = a[i];
      a[i] = a[next];
      a[next] = temp;
    }
  }
}
```

```
int (String::*compare_op)(const String);
compare_op = (order == Asc) ?
  String::operator < :  compare_op = String::operator >;
```

- `compare_op` a pointer to a method in `String`.
- That method has an argument of type `String`.
- Return value of the method is of type `int`
- Determine what the operator is, $<$ or $>$.

Friend Functions

- Data hiding is important in C++; private members can be accessed by the other members in the same class.
- But in some cases, it is convenient to allow a function to access the private member of a class.
- Can be achieved by declaring the function as a friend of the class. The friend has the following properties
  - Its prototype is placed inside the class definition and preceded by the `friend` keyword.
  - It is defined outside the class as a normal, non-member function.
  - It is called a normal non-member function.
- A function is a friend of two or more different classes, each class must contain the friend function prototype.

- Friendship is not
  - Reverse (symmetric), A is a friend of B does not imply B is a friend of A.
  - Transitive, A a friend of B, B a friend of C, does not imply A a friend of C.
  - inherited.
- Forward declaration
- A friend function can be a member function of one class and a friend of another.
- friend should not be abused.
- Prog5_6.

Program 5_6

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
const double PI=3.14159265;

class Capacitor; //Forward declaration
```

```
class Inductor {
  double inductance;
  double frequency;
  double reactance;
public:
  Inductor(double i, double f) {
    inductance=i;
    frequency=f;
    reactance=2*PI*frequency*inductance;
  }
  friend void compReactance(Inductor, Capacitor);
  friend double resFrequency(Inductor, Capacitor);
};
```

```
class Capacitor {
  double capacitance;
  double frequency;
  double reactance;
public:
  Capacitor(double c, double f)
  {
    capacitance=c;
    frequency=f;
    reactance=1/(2*PI*frequency*capacitance);
  }
  friend void compReactance(Inductor, Capacitor);
  friend double resFrequency(Inductor, Capacitor);
};
```

```
void compReactance(Inductor in, Capacitor cp)
{
 if (in.reactance>cp.reactance)
 {
  cout<<"Inductive reactance is greater that capacitive ";
  cout<<"reactance."<<endl;
 }
 else if(in.reactance<cp.reactance)
 {
  cout<<"Capacitive reactance is greater than inductive ";
  cout<<"reactance."<<endl;
 }
 else
  cout<<"Inductive and capacitive reactances are equal.\n";
}
```

```
double resFrequency(Inductor in, Capacitor cp)
{
 return 1/(2*PI*sqrt(in.inductance*cp.capacitance));
}

int main(int argc, char * argv[])
{
 Inductor ind(0.01,200);
 Capacitor cap (0.005,200);
 compReactance(ind,cap);
 cout <<setiosflags(ios::fixed)<<setprecision(4);
 cout<<"\n\nResonant Frequency = "<<resFrequency(ind,cap);
 cout<<" Hz\n\n";
 return 0;
}
```

- An entire class can be a friend of another class.

```
class A;
class B {
  //private members of B
public:
  friend A;
  //public members of B
};
```

- `friend A;` can be placed in the private section of the B class.
- Prog5_7

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
const double PI=3.14159265;
```

```
class Capacitor; //Forward declaration
class Inductor {
  double inductance;
  double frequency;
  double reactance;
public:
  Inductor(double i, double f) {
    inductance=i;
    frequency=f;
    reactance=2*PI*frequency*inductance;
  }
  friend class Capacitor;
};
```

```
class Capacitor {
  double capacitance;
  double frequency;
  double reactance;
public:
  Capacitor(double c, double f)
  {
    capacitance=c;
    frequency=f;
    reactance=1/(2*PI*frequency*capacitance);
  }
  void compReactance(Inductor); //
  double resFrequency(Inductor); //
};
```

```
void Capacitor::compReactance(Inductor in) //
{
  if (in.reactance>reactance)
  {
    cout<<"Inductive reactance is greater that capacitive "
    cout<<"reactance."<<endl;
  }
  else if(in.reactance<reactance)
  {
    cout<<"Capacitive reactance is greater than inductive "
    cout<<"reactance."<<endl;
  }
  else
    cout<<"Inductive and capacitive reactances are equal.\n
}
```

```
double Capacitor::resFrequency(Inductor in) //
{
 return 1/(2*PI*sqrt(in.inductance*capacitance));
}
int main(int argc, char * argv[])
{
 Inductor ind(0.01,200);
 Capacitor cap (0.005,200);
 cap.compReactance(ind);
 cout<<setiosflags(ios::fixed)<<setprecision(4);
 cout<<"\n\nResonant Frequency = "<<cap.resFrequency(ind);
 cout<<" Hz\n\n";
 return 0;
}
```

```
class F{
  int adm; //a data member
public:
  int f();
};

class C {
  int cdm;
public:
  int m(); //a method
  friend int f(); // toplevel friend
  firned int F::f(); // method friend
};
```

- if all the methods in `F` need to access private member of `C`

```
class C {
  friend F;
  int cdm;
public:
  int m();
  friend int t();
}
```

- A binary search tree class
  - a binary tree (including an empty tree).
  - a node has left and right subtree.
  - nodes in the left subtree have smaller keys than the key of the node.
  - two classes
    - A class `Node`
    - A class `BST` that is a friend of `Node`

```
const char None = ' ';

class BST; // forward declaration

class Node{
  friend BST;
  char val;
  Node* lc;
  Node* rc;
public:
  Node();
  int empty();
  void write();
};
```

```
class BST {
  Node* root;
  Node* tree;
  void addNodeAux(const char);
  void inorderAux(Node*);
public:
  BST();
  void addNode(const char);
  void inorder();
};
```

```
Node::Node()
{lc=rc=0; val=None;}

int Node::empty()
{ return val == None;}

void Node::write()
{cout << val;}

BST::BST()
{ root=tree=new Node;}

void BST::addNode(const char v)
{
  tree=root;
  addNodeAux(v);
}
```

```
void BST::addNodeAux(const char v)
{
  if (tree->empty()) {
    tree -> val = v;
    tree -> lc = new Node;
    tree -> rc = mew Node;
    return;
  }
  if (v<=tree->val)
    tree = tree -> lc;
  else
    tree = tree -> rc;
  addNodeAux(v);
}
```

```
void BST::inorder()
{ inorderAux(root);}

void BST::inorderAux(Node* n)
{
  if (n-> empty())
    return;
  inorderAux(n->lc);
  n-> write();
  inorderAux(n->rc);
}
```

- Static data member
  - A data member does not belong to any particular object of the class.
  - It belongs to the class.
  - A memory space is allocated to the static data member.
  - No matter how many class objects are instantiated, all objects of the class share the same copy of a static member.

```
class Ampifier {
  float gain, volatge;
public:
  static int acount;
  void cals;
}
```

- Suppose 10 `Ampilfier` are initiated, there are 10 `gains`, 10 `volatges`, but just only one copy of `acount`.
- To define a static data member outside the class, `data_type class_name::stvar_name = initial_value`

- `Amplifier` example, `int Amplifier::acount=1;`
- All static data members are initialized to 0 by default.
- Static data member can be accessed by a non-member function, but class name and the `::` operator are needed

```
void fun()
{
  Amplifier::acount ++;
}
```

- A `Potato` class, each potato has a `weight`, each time a `Potato` is created, total weight increases.
- We can design a static data member, `TotalWeight`, and update the `TotalWeight` in the constructor.
- Prog5_8

Program 5_8

```
#include <iostream>
using namespace std;
class Node {
  int num;
public:
  static int ncount;
    //track the number of nodes instantiated
  static int pcount;
    //track the number of nodes that used the printer
  Node() {num=1;ncount++;}
  Node(int x) {num=x; ncount++;}
  ~ Node() {ncount--;}
  void printer();
};
```

```
void Node::printer()
{
  pcount++;
  cout<<"\nThere are "<<ncount<<" nodes,"<<endl;
  cout<<"\t\t\tNode #"<<num<<" uses printer now!"<<endl;
  cout<<pcount<<" node(s) used printer so far,"<<endl;
}

int Node::ncount=0;
int Node::pcount=0;
```

```
int main(int argc, char * argv[])
{
  cout<<"\nThere are "<<Node::ncount<<" nodes."<<endl;
  Node n1, n2(2), n3(3);
  n2.printer();
  {
    Node n4(4), n5(5);
    n5.printer();
  }
  cout<<"\nThere are "<<Node::ncount<<" nodes."<<endl<<endl
  return 0;
}
```

- A member function can be declared `static`.
- A static member function is not attached to any object. It does not belong to an object but to the whole class.
- To call a static function, a class name followed by the scope resolution operator (`::`).
- A static member function does not have direct access to the private class data member. A pointer or a reference to an object should be passed to the function to enable access to the object's private data.
- A static member function does not have a `this` pointer.
- Prog5_9.

Program 5_9

```
#include <iostream>
using namespace std;
class Node {
  int num;
public:
  static int ncount;
    //track the number of nodes instantiated
  static int pcount;
    //track the number of nodes that used the printer
  Node() {num=1;ncount++;}
  Node(int x) {num=x; ncount++;}
  ~ Node() {ncount--;}
  static void printer(Node &n); //MODIFIED
};
```

```
void Node::printer(Node &n)
{
  pcount++;
  cout<<"\nThere are "<<ncount<<" nodes,"<<endl;
  cout<<"\t\t\tNode #"<<n.num<<" uses printer now!"<<endl;
  cout<<pcount<<" node(s) used printer so far,"<<endl;
}
int Node::ncount=0;
int Node::pcount=0;
```

```
int main(int argc, char * argv[])
{
  cout<<"\nThere are "<<Node::ncount<<" nodes."<<endl;
  Node n1, n2(2), n3(3);
  Node::printer(n2); //MODIFIED
  {
    Node n4(4), n5(5);
    Node::printer(n5); //MODIFIED
  }
  cout<<"\nThere are "<<Node::ncount<<" nodes."<<endl;
  return 0;
}
```

- The `this` pointer stores the address of an object used to call a non-static member function.

```
class Pixel {
  int x, y;
public:
  void set (int x, int y) {x=a; y=b;}
  void get() {cout <<x<<y;}
}
class Pixel {
  int x, y;
public:
  void set (int x, int y) {this->x=a; this->y=b;}
  void get() {cout <<this->x<<this->y;}
}
```

Prog5_10.

```
#include <iostream>
using namespace std;
class Pixel {
  int x,y;
public:
  Pixel() {cout<<"\t\tPixel created!"<<endl; x=0; y=0;}
  ~ Pixel() {cout<<"\t\tPixel destroyed!"<<endl;}
  void setCoord(int x1, int y1) {x=x1; y=y1;}
  void getCoord()
  {
    cout<<"Pixel's coordinate:"<<endl;
    cout<<"X="<<x<<" Y="<<y<<endl;
  }
  Pixel move_10()
  {
    x=x+10;
    y=y+10;
    return *this;
  }
};
```

```
int main(int argc, char* argv[])
{
  Pixel p1, p2;
  int x1, y1;
  cout<<"Enter X and Y Coordinates =>";
  cin>>x1>>y1;
  p1.setCoord(x1,y1);
  p1.getCoord();
  p2 = p1.move_10();
  p2.setCoord(5,6);
  p1.move_10();
  p2.getCoord();
  p1.getCoord();
  return 0;
}
```

- The object on which the member function is invoked is passed by the `this` pointer to the function.
- const member function: it cannot modify the object that is used to invoke the function.

```
// const member function prototype
return_type function_name(parameter list) const;

// const member function definition
return_type class_name::function_name
    (parameter list) const
{
  // body of the function
}
```

- If we don't expect that a method will change anything in the object, the method should be declared `const`.
- If an object is declared a `const`
- `const Power source (110,60);`, only `const` method can be called.

```
class Power {
  float voltage;
  float frequency;
public:
  Power(float v, float f)
    {voltage=v; frequency=f;}
  void display(){cout<<voltage<<frequency;}
  float getVolt(){return voltage;}
};
```

- Given the previous definition, the following code fragment contains syntax errors:

```
const Power source(110,60); //Correrct
Power bat1(12,0); //Correct
float v1, v2;
v1=source.getVolt(); //Error non-const function
  //cannot process const object
v2=bat1.getVolt; //Correct
source.display(); //Error
bat1.display(); // Correct
```

- That can be fixed if `display()` and `getVolt()` are defined as `const` function.

```
class Power {
  float voltage;
  float frequency;
public:
  Power(float v, float f)
    {voltage=v; frequency=f;}
  void display() const {cout<<voltage<<frequency;}
  float getVolt() const {return voltage;}
};
```

- It is good programming practice to declare all member functions that do not modify objects as `const`.